Self-Steering: A Small Language Model That Switches Its Own Neuron and Weight Edits Mid-Conversation

Nirav Surabhi
https://github.com/21J3phy/self-steer

November 2026

Abstract

Small edits to a language model's MLP layers can switch behaviors on and off, but they are usually applied statically, to every input. We build **self-steer**, a harness in which Qwen3.5-4B switches such edits on itself, mid-conversation, through an ordinary tool call. Every edit is a bias patch or a rank-1 weight patch on a layer's MLP output, kept next to the frozen 8-bit weights. When the model lacks an edit it needs, it can ask an outside controller (Claude, over the Model Context Protocol) to build one from a few contrastive examples.

The runtime reproduces a published steering result token for token (16/16 generations). Switching costs microseconds and does not slow decoding. Conversations survive mid-stream switches: recall was 24/24, and the steered cache differs from a clean one by a mean KL of 0.0004 nats.

In a pre-registered, blind-graded test on 96 sealed questions × 2 seeds, we compared deciders for when to apply a length-reducing edit:

- **Claude, routing on question text alone:** answers were 20% shorter with unchanged core accuracy (+0.5 pts, 95% CI [-2.1, +3.1]).
- **The 4B model, deciding for itself:** answers were 68% shorter but lost 13 points (CI [-20.8, -5.2]).
- **A "be concise" prompt:** lost 29 points.

A placebo control traces the self-selection failure to the edit compounding with the tool-use context: the context alone shortened answers 61% at a non-significant cost, and the edit on top cost a further -9.4 points.

In a live demonstration, Qwen asked Claude for a mode for explaining things to a five-year-old, and Claude built and returned it in 15.6 s. On held-out prompts, one of four Claude-built modes clearly worked, and prompting matched or beat the edits for style. Switching edits live is cheap and safe for memory; the hard part is deciding when to switch.

1 Introduction

Interpretability work keeps finding small, legible handles inside language models: single neurons, directions, and low-rank edits that turn behaviors up or down [12, 7, 13]. Most of these handles are used statically. A researcher picks an edit, applies it to every input, and measures the average effect.

Our starting point is such an edit. Lazy Genius Qwen [4] is a recipe of 344 MLP neuron doses, applied only to the first six answer positions of Qwen3.5-4B. It makes answers about 37% shorter. In two pre-registered, blind-graded tests, core accuracy stayed within a point of the unedited model. But the average hides structure. The edit helps code (Python answers 77% shorter and slightly *more* accurate) and hurts multi-step planning and scheduling, where the terse model takes greedy shortcuts. An always-on edit is the wrong unit. The edit should be on for some requests and off for others.

This paper asks who should flip the switch, and whether the switch can be flipped live, in the middle of a conversation, without retraining, reloading, or losing the conversation. We build **self-steer**, a harness in which:

1. every edit is a bias patch or a rank-1 weight patch on a layer's MLP output projection, kept next to the frozen 8-bit weights and switchable between any two tokens;
2. the model itself can switch its edits through an ordinary tool call (`set_mode`);
3. when no edit fits, the model can ask an outside controller (Claude, over the Model Context Protocol) to build one (`request_mode`). The controller writes a few contrastive examples, which become a new edit in seconds.

Our contributions are:

- **A faithful runtime for live edits.** It reproduces published Lazy Genius generations token for token (16 of 16 cells). Switching costs microseconds, and decode speed is unchanged (§4.1, §4.2).
- **Evidence that switching mid-conversation does not break memory**, and a measurement of how much a steered cache differs from a clean one (§4.3).
- **A pre-registered test of who should decide** (§4.4).
  - *Claude routing on the question text alone* kept accuracy (+0.5 pts core, 95% CI [-2.1, +3.1]) with 20% fewer words.
  - *The 4B model deciding for itself* cut 68% of words but lost 13 points of core accuracy.
  - *A "be concise" prompt* lost 29 points.

  A placebo control shows why self-selection fails: the edit compounds with the tool-use context.
- **A case study of a frontier model building new edits** for a small model over MCP, including a weight edit, evaluated on held-out prompts against prompting (§4.5).

2 Related work

Activation steering and weight editing. ActAdd [12] and contrastive activation addition (CAA) [7] steer with mean activation differences; representation engineering [13] reads and controls high-level concepts. CAST [3] conditions a steering vector on the model's own activations, so the input decides whether a fixed edit applies. Weight-Space Programming [11] has a frontier model read raw weights and edit a small model directly. It fixed Qwen3.5-0.8B's reasoning loops by removing one direction from 64 MLP output columns in each of two layers (0.016% of weights), offline, with a budget of forward passes. Our `ablate` edits are the same kind of rank-1 weight edit, applied live and reversibly.

Self-adaptive models. Transformer^2 [10] runs a first pass to identify the task, then a second pass with mixed singular-value expert vectors. SEAL [14] has the model write its own fine-tuning data and update its weights. Text-to-LoRA and Doc-to-LoRA [9] generate adapters from text with a hypernetwork. WeAct (reported October 2026; we could not locate the paper) lets a Qwen3-32B agent load skill LoRAs through tool calls. These methods either need trained adapters or experts, or update weights slowly and permanently. self-steer uses small, interpretable edits that switch per token, and lets a frontier model mint new ones on demand.

Steering and the cache. Steered states written into the KV cache persist. Kang et al. [2] identify cache contamination as a failure mode of residual steering in multi-turn dialogue. KV-cache steering [1] edits the cache once after prefill. We measure the effect for switched edits directly, and offer re-encoding as a policy.

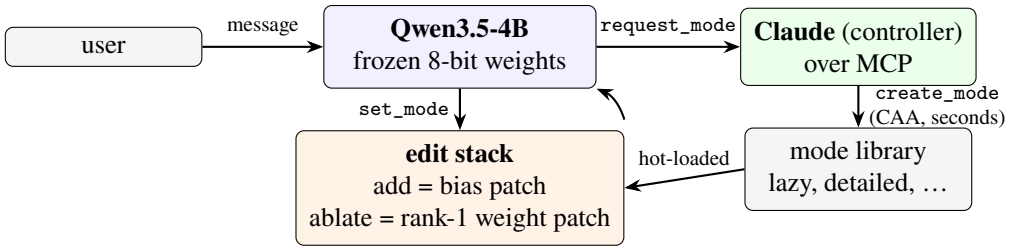


Figure 1: Self-steering. Qwen switches edits on its own MLP layers through a tool call. Each edit is a bias or rank-1 patch next to the frozen weights, live from the next token. When no mode fits, Qwen asks the controller (Claude, over MCP), which builds one from a few contrastive examples and hands it back.

3 System

Model. We use Qwen3.5-4B, 8-bit affine quantization in MLX (`mlx-community/Qwen3.5-4B-8bit`, revision `1704583a`), in non-thinking mode with Qwen's recommended sampling: temperature 0.7, top-p 0.8, top-k 20, presence penalty 1.5, and at most 1024 new tokens per turn. The model has 32 decoder layers. Every fourth layer is full attention (a KV cache), and the other 24 are Gated DeltaNet linear-attention layers (a recurrent state). Each layer has a dense SwiGLU MLP with 9216 neurons, so there are 294,912 MLP neurons in total.

Edits are patches on `down_proj`. Each layer's MLP output $o_\ell = W_\ell^{\text{down}}\, a_\ell$ is added directly to the residual stream. We wrap every `down_proj` once and support two kinds of edit, both applied in float32 and cast back once:

- **add**: $o_\ell \leftarrow o_\ell + v_\ell$. This is exactly a bias patch on W_ℓ^{down}. The Lazy Genius recipe is of this kind: $v_\ell = \sum_n d_n\, m_\ell\, w_{\ell,n}/\|w_{\ell,n}\|$, summed over its 344 (layer, neuron, dose) entries, where $w_{\ell,n}$ is the neuron's write column and m_ℓ the median residual norm at layer ℓ. Vectors built from contrastive activation differences (CAA) are also of this kind.
- **ablate**: $o_\ell \leftarrow o_\ell - (o_\ell \cdot u_\ell)\, u_\ell$ for a unit direction u_ℓ. This equals the rank-1 weight edit $W_\ell^{\text{down}} \leftarrow (I - u_\ell u_\ell^\top) W_\ell^{\text{down}}$. It is the same kind of edit Weight-Space Programming applied by hand [11], here applied next to the frozen quantized weights instead of rewriting them.

An edit can carry a *window*: it is active only for the first N answer positions, counted from the position that predicts the first answer token. Lazy Genius uses N=6.

Switching between tokens. The decode loop tells the edit stack the absolute positions of each forward pass. MLX builds its graph when the model is called from Python, so an edit changed between two forward passes applies to the very next prediction, with no reload and no recompilation. We wrote our own prefill and decode loop on a persistent cache rather than using `stream_generate`, for two reasons. `stream_generate` computes one token ahead, so an edit changed after token n would first affect token n+2. It also resets the penalty history on every call. The loop reproduces `mlx_lm` 0.31.3's prefill chunking, logits processors and Transformers-order sampler exactly (§4.1).

Sessions and memory. A session keeps an append-only *token ledger*: exactly the tokens the model has read or written, including its own tool calls and the tool responses. New turns append rendered suffixes in Qwen's chat format instead of re-rendering the whole conversation, so the cache stays valid. Qwen's template drops earlier turns' empty `<think>` blocks when a chat is re-rendered, so re-rendering would invalidate the cache on every turn. After an edit has written steered states into the cache, the session follows one of two policies:

- `keep`: leave the steered states in place. This costs nothing.
- `reprefill`: before the next user turn, re-encode the whole ledger with no edits, so the model's memory of the conversation is the unedited model's.

The linear-attention state cannot be trimmed, so sessions can also be snapshotted and forked with a deep copy of the cache.

The model's tools. Qwen sees a `set_mode(name, scope)` tool and a system prompt listing its modes, each with a one-line description of when it helps. When the decoder samples `</tool_call>`, the session parses the call with `mlx_lm`'s Qwen3-Coder parser and arms the edit for the next answer (`scope=answer`) or until changed (`conversation`). It then appends the tool response and lets Qwen continue. Scopes are enforced by the harness, not by the model. This matters because a model in lazy mode might never decide to switch lazy mode off. With a controller connected, Qwen also sees `request_mode(description)`, which asks the controller to build a mode it does not have.

The controller (Claude over MCP). One process holds the model on a single worker thread. It exposes an MCP endpoint and a web chat. The controller tools are:

- `wait_for_request` and `fulfill_request`;
- `create_mode` (from contrastive examples) and `try_mode` (normal vs. mode, side by side);
- `set_mode`, `chat`, `save_mode`, `status` and `transcript`.

While Qwen waits on a request, the worker keeps processing other jobs, so the controller's `create_mode` and `try_mode` calls can use the model without deadlock. The controller in this paper is Claude (Opus 5.5) running in Claude Code.

Building a mode in seconds (CAA). `create_mode` takes K triples (prompt, positive answer, negative answer). For each answer we run the chat-formatted prompt plus answer through the unedited model and average the residual stream over the answer tokens after every layer. The direction at layer ℓ is $\delta_\ell = \overline{h_\ell^+} - \overline{h_\ell^-}$, averaged over the examples. An **add** mode uses $v_\ell = s\, m_\ell\, \delta_\ell / \|\delta_\ell\|$ on a chosen band of layers (8–16 by default), with strength s in the same dose units as Lazy Genius. An **ablate** mode removes $\delta_\ell / \|\delta_\ell\|$ from those layers' MLP writes, which is a rank-1 weight edit.

4 Experiments

All experiments were pre-registered in `experiments/PREREGISTRATION.md` (committed at `68bcf54` before any E2–E5 generation), except the placebo condition in E4. That condition was added after seeing the E4 results and is labeled exploratory. They ran on one Apple M2 Pro laptop with 16 GB of memory, one model process at a time.

4.1 E1: The runtime reproduces Lazy Genius exactly

We replayed published Lazy Genius test-2 cells [4] through the new engine: eight questions spanning all six categories, at seed 17, both unedited and with the `lazy` mode armed for the answer. 16 of 16 generations were token-identical to the published ones, including prompts, all six steered positions, and answers of up to 695 tokens. The engine therefore applies the same edit with the same sampling. This also lets E4 reuse the published, already-graded generations as its `baseline` and `lazy` conditions.

Operation	Median cost
Decode, no edit / `lazy` (6 pos.) / whole-answer add / 9-layer ablate	27.0 / 24.8 / 29.8 / 28.3 tok/s
Switch an edit on and off	1 μs
Build a CAA mode from 2 / 8 examples	1.2 / 3.9 s
Re-encode 512 / 1024 / 2048 / 4096 tokens	2.3 / 4.6 / 8.7 / 19.9 s
Snapshot the cache (2048 tokens)	0.2 ms

Table 1: Overhead on an M2 Pro (16 GB), Qwen3.5-4B 8-bit.

4.2 E2: Switching is free

Table 1 shows the measured costs.

- **Decode speed.** With greedy decoding (median of three 256-token runs), speed is the same with no edit (27.0 tok/s), with Lazy Genius over the whole answer (29.8), with a 9-layer rank-1 ablation (28.3), and with both (28.5). The run-to-run spread on a shared laptop (11–30 tok/s) is larger than any difference between edits.
- **Switching** an edit on or off costs 1 μs of Python.
- **Building a mode** from 8 contrastive examples takes 3.9 s, or 1.2 s from 2 examples.
- **Re-encoding a conversation** with no edits runs at about 236 tok/s: 4.6 s for 1k tokens and 19.9 s for 4k.
- **A cache snapshot** (deep copy) takes 0.2 ms.

4.3 E3: Switching mid-conversation does not break memory

Setup. We generated 24 three-turn conversations from fixed templates.

1. **Turn 1** gives four facts: a locker code, a manager's name, a gate and a dog's name, ending with "Just reply 'Noted.' ".
2. **Turn 2** asks an unrelated everyday question, such as "What does HTTP status 404 mean?". The controller arms an edit for this answer only.
3. **Turn 3** asks for two of the facts, with no edit.

The turn-2 edits were `lazy` (6 positions) and `full`, the same vector applied over the entire turn-2 answer. `full` is a deliberately heavy edit that cut turn-2 answers from 176 to 25 words. After turn 2 each conversation was forked: `keep` kept the steered cache, and `reprefill` re-encoded the whole conversation with no edits.

Results. Recall was perfect everywhere:

- no edit: 24/24;
- `lazy`: 24/24 with `keep` and 24/24 with `reprefill`;
- `full`: 24/24 with `keep` and 24/24 with `reprefill`.

To see how much a steered cache changes the model's next-token beliefs, we compared the `keep` cache with a clean re-encoding of the same conversation. We fed both the turn-3 prompt and the first 32 tokens of the clean branch's answer, and measured KL(clean || keep) at each prediction position:

- **`lazy`:** mean KL 0.0004 nats (max 0.005), and the two caches agree on the top token 99.3% of the time.
- **`full`:** mean KL is larger, 0.0020 nats (max 0.030), with 99.2% agreement. This supports H3b.

Steered states do leave a trace in the cache, consistent with [2], but at this scale it is far below what changes an answer. Re-encoding the conversation took 1.3 s on average. We use `keep` as the default and offer `reprefill` for long sessions with heavy edits.

Condition	Words	Fewer words	Core	Δ core (pts, 95% CI)	Δ whole (pts, 95% CI)	Chose lazy
Normal Qwen	207	–	168/192	–	–	–
Lazy Genius, always on	136	34%	168/192	+0.0 [-4.2, +4.2]	+7.8 [+1.0, +15.1]	192
Qwen decides (`self`)	66	68%	143/192	-13.0 [-20.8, -5.2]	-4.2 [-14.1, +5.7]	118
placebo: tool, no edit[†]	80	61%	161/192	-3.6 [-10.4, +3.1]	-1.0 [-10.4, +8.9]	(118)
"Be concise" prompt	15	93%	112/192	-29.2 [-38.5, -19.8]	-15.6 [-27.6, -3.6]	–
Claude routes (`routed`)	166	20%	169/192	+0.5 [-2.1, +3.1]	+5.2 [+0.0, +10.9]	130
Hindsight oracle	145	30%	176/192	+4.2 [+1.6, +7.8]	+14.6 [+8.9, +20.8]	174

Table 2: E4 on 96 sealed questions × 2 seeds. "Words" is the mean words per answer, and "fewer words" is the total-word reduction vs. normal Qwen. Δ columns are paired bootstrap estimates over questions. [†]Exploratory, not pre-registered. Its "chose lazy" count is how often Qwen called `set_mode(lazy)`, which did nothing.

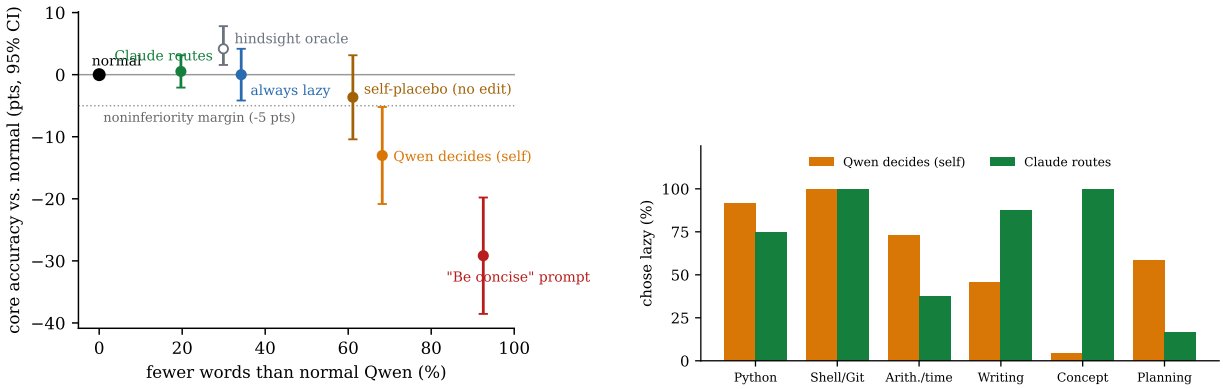


Figure 2: Left: length vs. accuracy for each policy. Right: how often each decider chose `lazy`, by question category.

4.4 E4: Who should decide when to be lazy?

Setup. We used the 96-question sealed set from Lazy Genius test 1, with six categories: Python, shell/Git, arithmetic and time, writing and reading, concepts, and planning. Each question was run at seeds 17 and 29, giving 192 (question, seed) cells per condition. The `baseline` and `lazy` (always on) answers are the published, blind-graded test-1 generations, which E1 shows the engine reproduces. The new conditions are:

- `self`: Qwen sees the `set_mode` tool, a library containing only `lazy`, and its description (§A), and decides per question.
- `prompt`: no edit, with the system prompt "Be concise. Give the answer directly, with no preamble, no restating the question, and no extra explanation unless asked."
- `routed`: a Claude subagent read only the 96 question texts and the same mode description, and labeled each question lazy or normal. The cell's answer is the existing `lazy` or `baseline` generation.
- `oracle`: per cell, the `lazy` answer if its grade is at least the baseline's. This is a hindsight upper bound for a lazy/normal switch, not a policy.

Grading. The `self` and `prompt` answers were shuffled with 48 previously graded answers as calibration. Nine blind Claude Opus 5.5 graders then each graded 48 answers, with Lazy Genius's per-item rubrics and grading guide. On the calibration items the new graders matched the original grades on 48/48 core and 46/48 whole-answer judgments.

Results (Table 2, Figure 2).

- **Lazy Genius, always on,** reproduced its published profile: 34% fewer words, core accuracy unchanged (+0.0 pts, CI [-4.2, +4.2]), and more fully correct answers (+7.8 pts).
- **Claude routing** kept accuracy, +0.5 pts core (CI [-2.1, +3.1]) and +5.2 pts whole, with 20% fewer words. It chose `lazy` for every shell and Git question but for only 4 of 24 planning cells.
- **The hindsight oracle** shows the headroom of a perfect lazy/normal switch: +4.2 pts core and +14.6 pts whole at 30% fewer words.
- **Qwen deciding for itself** failed pre-registered H4b and H4c. It produced the shortest edited answers (68% fewer words, so H4a passed), but lost 13 points of core accuracy (CI [-20.8, -5.2]). It chose `lazy` in 118 of 192 cells, including 14 of 24 planning cells, although the mode description warned against planning. Arithmetic and time questions fell from 41 to 24 of 48 correct. The typical failure was an answer stated first with no working, and the answer was wrong: "The total discount is 27%" instead of 37%, or "March 30" instead of March 31.
- **The "be concise" prompt** was worst: 93% fewer words and -29.2 pts core (CI [-38.5, -19.8]).

Why self-selection fails: a placebo control (exploratory). The `self` condition differs from `baseline` in two ways: a tool-use context (system prompt, tool call, tool response) and the edit. The placebo condition keeps the first and removes the second. Qwen sees the same tools and prompt, makes the same calls (it chose `lazy` in the same 118 cells, since the context is identical up to that point), and gets the same tool response, but no edit is armed. The tool-use context alone cut 61% of words at a small, non-significant accuracy cost (-3.6 pts, CI [-10.4, +3.1]). Adding the edit on top cost a further -9.4 points of core accuracy, `self` vs. placebo paired per question (CI [-14.6, -4.2]). In the 118 cells where Qwen chose `lazy`:

- **placebo:** answers averaged 58 words, with 94 correct;
- **with the edit:** 34 words and 77 correct;
- **the same edit without the tool context** (the `lazy` condition): 110 words and 102 correct, against 104 for normal Qwen.

The edit and the context compound. Lazy Genius was calibrated on a bare chat prompt, and after a tool call that already pushes toward brevity, the same edit pushes past the point where the model still shows its working. Self-steering needs edits calibrated in the context where they are switched on, or a decider that sits outside that context.

Grader consistency. Of the 96 cells where `self` and placebo produced byte-identical answers, graded in separate blind batches, the grades disagreed on 1.

4.5 E5: Claude builds new edits for Qwen over MCP

The request loop. In the live web chat, the user wrote: "Explain how vaccines work so my five-year-old would get it. If you do not have a mode for talking to little kids, ask Claude to build one first."

1. Qwen called `request_mode` with its own description of the mode it wanted ("Explain complex scientific concepts using simple analogies, short sentences, and engaging language suitable for a five-year-old child…").
2. Claude, the controller in this paper's Claude Code session, received the request through `wait_for_request`. It called `create_mode` with 8 contrastive examples it had written (kid-style vs. textbook answers to other questions), checked the result with one `try_mode`, and called `fulfill_request`.
3. Qwen's tool call returned. Qwen called `set_mode(eli5)` and answered under the new edit: a 170-word story about an "Immune Army" that remembers a germ's claw.

The request was fulfilled 15.6 s after Qwen asked, and the whole turn took 29 s (Figure 3).

That was the second run. In the first, Claude's initial strength (0.3) made Qwen degenerate in `try_mode`. Claude rebuilt the mode at 0.08 and fulfilled it after 66 s. Without the explicit request in the message, Qwen

Mode	Metric	Normal	Mode (edit)	Instruction (prompt)
`eli5`	Flesch–Kincaid grade ↓	10.5	3.7	3.3
	words	319	153	73
`bullets`	mostly-bullet answers (of 20)	17	13	20
`plain-math` (weight edit)	answers with LaTeX (of 20) ↓	20	20	13
	correct number present (of 20)	20	20	20
`plain-math-add`	answers with LaTeX (of 20) ↓	20	7	13
	correct number present (of 20)	20	11	20

Table 3: E5: held-out evaluation of modes Claude built over MCP, 20 prompts each, with a fixed seed per prompt.

did not ask for a new mode. It chose the built-in `detailed` mode and wrote a kid-friendly answer through prompting alone.

Held-out evaluation. Claude built three modes from 8 examples each.

- `eli5` and `bullets` are add edits on layers 8–16.
- `plain-math` is a rank-1 ablation of the "LaTeX" direction from all 32 layers' MLP writes, which is a weight edit. Ablating only layers 8–16 left the outputs unchanged in `try_mode`.
- `plain-math-add` is the same direction applied as a negative add edit on layers 8–16.

Strengths were set by Claude with `try_mode` on prompts outside the evaluation set. Each mode was evaluated on 20 held-out prompts against the unedited model and against a system-prompt instruction asking for the same style (Table 3).

Results: one clear success, one entangled edit, two misses.

- **`eli5` worked.** It cut the Flesch–Kincaid grade of answers from 10.5 to 3.7 and halved their length. That is close to the readability of simply asking for a five-year-old explanation (3.3), though the prompted answers were shorter still.
- **`plain-math-add` was entangled with digits.** It removed LaTeX from more answers than the instruction did (7 vs. 13 of 20 still had LaTeX). But only 11/20 answers still contained the correct number, against 20/20 for the instruction. The edit dropped or mangled digits: "46 km/h multiplied by ␣ hours". In the contrastive examples, numbers appear *inside* the LaTeX, so the "LaTeX direction" also encoded part of how numbers are written.
- **The weight edit `plain-math` did nothing on held-out prompts.** This is a rank-1 ablation of that direction from all 32 MLP layers, and it left LaTeX in 20/20 answers. Removing a direction only from the MLP writes leaves the attention path and the residual stream free to carry it. Weight-Space Programming succeeded with a similar edit after a search over which columns to edit [11]. Building one from 8 examples in seconds did not.
- **`bullets` failed.** Normal Qwen already answered 17/20 of these how-to prompts mostly in bullets. The edit produced 13/20, and the instruction 20/20.

For style, prompting was as good as or better than a quickly built edit. What the loop does show is that a frontier model can turn a request into a live, measurable edit in seconds, and that `try_mode` catches broken strengths. Building edits that beat prompting will need better examples, held-out checks inside the loop, and probably edits on the attention path as well.

5 Discussion

The mechanics are easy. The judgment is hard. Live edits cost nothing measurable. They leave the model's memory intact, and a frontier model can mint a usable new edit from eight examples in seconds. The open problem is the policy.

- **Claude decided well from the question text alone.** It chose `lazy` for every shell and Git question but only 4 of 24 planning cells, and kept accuracy at a fifth fewer words.
- **The 4B model, deciding for itself, did not.** It chose `lazy` on 14 of 24 planning cells despite a description telling it not to. More importantly, choosing an edit through a tool call is not neutral: the edit stacks on top of the tool-use context (§4.4).

An edit calibrated in one context (a bare chat prompt) is not the same edit in another (after a tool call and tool response). Self-steering systems will need either context-aware calibration or a controller that sits outside the model's own context.

Prompting is not a substitute for small edits, nor always the reverse. A one-line "be concise" system prompt shortened answers far more than any edit, and it cost 29 points of accuracy, mostly by suppressing the working needed for arithmetic and scheduling. For style requests (§4.5), prompting was competitive with or better than the CAA edits Claude built. Edits earn their place when a behavior needs to be switched precisely, measured, or reversed, and when the prompt channel is unavailable or untrusted.

Safety. A model that can change its own weights raises obvious concerns. Activation steering itself can degrade safety behavior [8]. In self-steer the model can only switch edits from a library, the harness enforces their scope, and new edits come from an outside controller that can refuse. All edits are logged with token positions and can be reverted. We did not test adversarial use.

6 Limitations

- **Scope.** We tested one model (Qwen3.5-4B), at one precision (MLX 8-bit), on one laptop. Test sets are small: 96 questions $\times$ 2 seeds for E4, 24 conversations for E3, and 20 prompts per mode for E5.
- **Grading.** Graders are LLMs (Claude Opus 5.5), blind to condition, and calibrated against earlier grades. The controller is also Claude, so we used separate instances that had no access to each other's outputs.
- **Reused generations.** E4's baseline and always-on conditions are generations from an earlier study. E1 shows the engine reproduces them exactly, but they were graded in a separate batch.
- **The planning warning.** The `lazy` description's warning about planning came from that study's second test, on different questions.
- **The demo.** The E5 request loop is a demonstration. The user's message asked for a new mode explicitly, and the controller was a human-supervised Claude Code session, not an autonomous service.
- **CAA modes are coarse.** Strengths that are too high break the model, and the useful range is narrow (roughly 0.06–0.1 per layer here).

7 Conclusion

Small edits to a language model's MLP layers can be switched live, between any two tokens, at no measurable cost and without disturbing the conversation. A frontier model can build new ones on demand. Whether such a system helps depends on who holds the switch. In our tests a frontier controller held it well. The model holding its own switch over-applied it and lost accuracy. Making small models better judges of their own edits, or calibrating edits to the context they are switched in, is the next step.

AI assistance. The harness, experiments and analysis were built with Claude Code (Claude Opus 5.5), which also acted as the controller in E5. Answers were graded by separate blind Claude Opus 5.5 instances.

References

[1] M. Belitsky et al. KV cache steering for controlling frozen LLMs. arXiv:2507.08799, 2025.

[2] D. Kang, Z. Liu, N. Ma, Y. Huang, Z. Tan, and M. Jiang. Prompt-activation duality: Improving activation steering via attention-level interventions. arXiv:2605.10664, 2026.

[3] B. W. Lee et al. Programming refusal with conditional activation steering. In *ICLR*, 2025. arXiv:2409.05907.

[4] N. Surabhi. Lazy Genius Qwen: steering 344 neurons for the first 6 answer tokens. https://github.com/21J3phy/lazy-genius-qwen, 2026.

[5] Model Context Protocol. https://modelcontextprotocol.io, 2024–2026.

[6] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert. MLX: Efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023.

[7] N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner. Steering Llama 2 via contrastive activation addition. arXiv:2312.06681, 2023.

[8] The rogue scalpel: Activation steering compromises LLM safety. arXiv:2509.22067, 2025.

[9] Sakana AI. Instant LLM updates with Doc-to-LoRA and Text-to-LoRA. arXiv:2602.15902, 2026.

[10] Q. Sun, E. Cetin, and Y. Tang. Transformer-Squared: Self-adaptive LLMs. arXiv:2501.06252, 2025.

[11] R. Takehi, J. Teoh, A. Zhang, and O. Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. https://rikiyat.github.io/blog/weight-space-programming/, November 2026.

[12] A. M. Turner et al. Steering language models with activation engineering. arXiv:2308.10248, 2023.

[13] A. Zou et al. Representation engineering: A top-down approach to AI transparency. arXiv:2310.01405, 2023.

[14] A. Zweiger, J. Pari, H. Guo, E. Akyürek, Y. Kim, and P. Agrawal. Self-adapting language models. In *NeurIPS*, 2025. arXiv:2506.10943.

A Prompts and tool definitions

System prompt in the `self` and placebo conditions. This text follows Qwen's own tool block, which lists the `set_mode` JSON schema:

```
You can change how your own neurons are set before you answer, by calling
set_mode. Each mode is a small edit to your MLP layers. Your modes:
- lazy: Short, direct answers with no padding. Good for quick facts,
definitions, code snippets, one-line commands and simple arithmetic. Avoid it
for multi-step planning or scheduling puzzles, where it tends to take greedy
shortcuts.
- normal: no edit.
Before answering, decide whether a mode clearly fits the request. If one does,
call set_mode once, then answer after the tool result. If none fits, just
answer normally without calling any tool.
```

With a controller connected, one more line is added: "If the user wants a style none of your modes gives, you may call request_mode to ask Claude to build one, then switch it on with set_mode."

Tool result after `set_mode`. "Mode 'lazy' is on for your next answer. Now answer the user." The placebo condition returns the same text but arms no edit.

The `lazy` description and the planning warning. The warning against planning and scheduling comes from Lazy Genius test 2, a different question set from the test-1 set used in E4.

Router brief (`routed`). A Claude subagent was told the goal ("shorter answers without hurting correctness"), given the mode description above, and allowed to read only a file of the 96 question texts, with no rubrics, answers or grades. It labeled 65 questions lazy and 31 normal.

Grader brief. Each grader read only Lazy Genius's `GRADING_SAMPLED.md` and one blind chunk. It could run tiny isolated code fixtures, and it returned `core_correct`, `correct`, `completion_failure` and a reason for each item. The Lazy Genius tests ran graders at a "high" effort setting. Here they ran at the default setting, which is a deviation. The calibration items (§4.4) measure its effect.

B Deviations from the pre-registration

- **Placebo condition (E4).** It was added after the E4 results and is exploratory.
- **Grader effort setting.** The default was used instead of "high"; see above.
- **Decode benchmark (E2).** It used greedy decoding, so each edit produced different text. Speeds are per token.

C Reproducing

The code, presets, raw generations, blind keys, judgments and analysis outputs are in the repository. All paper numbers are generated by `paper/make_numbers.py` from `results/*.json`.

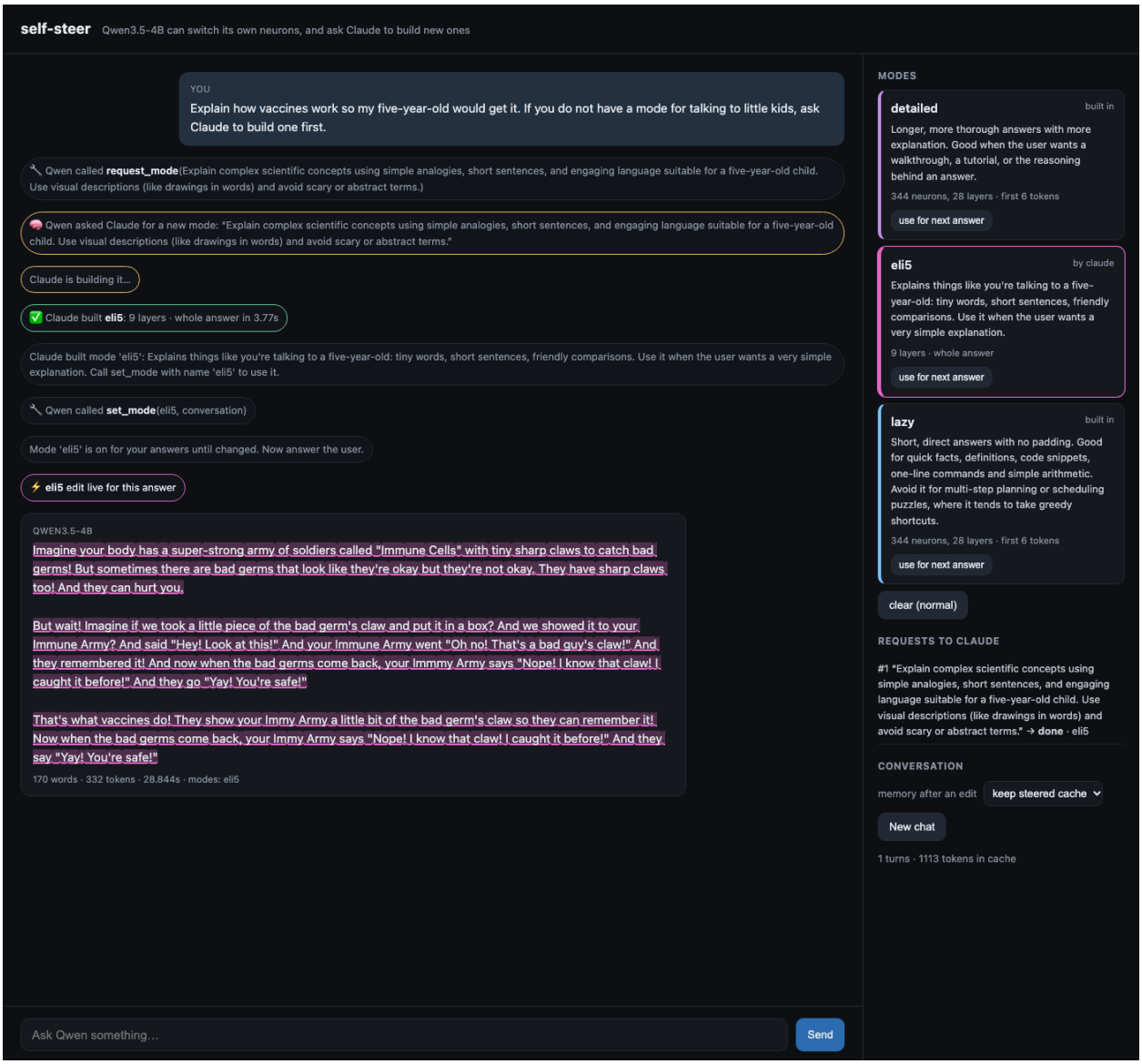


Figure 3: The web chat during run 2 of the request loop. Qwen asks Claude for a mode it does not have (yellow), Claude builds and returns `eli5` (green), and Qwen switches it on and answers. Steered tokens are highlighted.